

FullStack.Cafe - Kill Your Tech Interview

Q1: What is ASP.NET MVC? ☆

Topics: ASP.NET MVC

Answer:

ASP.NET MVC is a web application Framework. It is light weight and highly testable Framework. MVC separates application into three components—Model, View and Controller.

Q2: What is *Layout* in MVC? ☆

Topics: ASP.NET MVC

Answer:

Layout pages are similar to master pages in traditional web forms. This is used to **set the common look across multiple pages**. In each child page we can find

```
@{
    Layout = "~/Views/Shared/TestLayout1.cshtml";
}
```

This indicates child page uses `TestLayout` page as it's master page.

Q3: What is Razor Pages? ☆☆

Topics: ASP.NET MVC

Answer:

[Razor Pages](#) is a new feature of ASP.NET Core that makes coding page-focused scenarios easier and more productive.

With Razor Pages, you have this one Razor file (*.cshtml*), and the code for a single page lives inside of that file, and that file also represents the URL structure of the app. Therefore, you got everything inside of one file, and it just works.

However, you can separate your code to the *code behind* file with *.cshtml.cs* extension. You would usually have your view model and handlers (like action methods in MVC) in that file and handle the logic there. Of course, you could also have your view model moved to separate place.

Since Razor Pages is part of the MVC stack, you can use anything that comes with MVC inside of our Razor Pages.

Q4: Can you explain *Model, Controller* and *View* in MVC? ☆☆☆

Topics: ASP.NET MVC

Answer:

- **Model**—It's a business entity and it is used to represent the application data.
- **Controller**—Request sent by the user always scatters through controller and it's responsibility is to redirect to the specific view using View() method.
- **View**—It's the presentation layer of MVC.

Q5: What are the advantages of MVC over ASP.NET? ☆☆

Topics: ASP.NET MVC

Answer:

- Provides a clean separation of concerns among UI (Presentation layer), model (Transfer objects/Domain Objects/Entities) and Business Logic (Controller).
- Easy to UNIT Test
- Improved reusability of model and views. We can have multiple views which can point to the same model and vice versa.
- Improved structuring of the code

Q6: What is *Razor View Engine*? ☆☆

Topics: ASP.NET MVC

Answer:

Razor is the first major update to render HTML in MVC 3. Razor was designed specifically for view engine syntax. Main focus of this would be to simplify and code-focused templating for HTML generation. Below is the sample of using Razor:

```
@model MvcMusicStore.Models.Customer
@{ViewBag.Title = "Get Customers";}
<div class="cust"> <h3><em>@Model.CustomerName</em> </h3>
```

Q7: What is the use of `ViewModel` in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

`ViewModel` is a plain class with properties, which is used to *bind* it to strongly typed **view**. `ViewModel` can have the validation rules defined for its properties using *data annotations*.

Q8: What you mean by Routing in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

Routing is a pattern matching mechanism of incoming requests to the URL patterns which are registered in route table. Class—"UrlRoutingModule" is used for the same process.

Q9: What are *Actions* in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

Actions are the methods in `Controller` class which is **responsible for returning the view or json data**. Action will mainly have return type— `ActionResult` and it will be invoked from method `InvokeAction()` called by controller.

Q10: Explain `Bundle.Config` in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

“**BundleConfig.cs**” in MVC is used to register the bundles by the bundling and minification system. Many bundles are added by default including jQuery libraries like— `jquery.validate`, `Modernizr`, and default CSS references.

Q11: Explain Sections in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

Section are the part of HTML which is to be rendered in layout page. In Layout page we will use the below syntax for rendering the HTML –

```
@RenderSection("TestSection")
```

And in child pages we are defining these sections as shown below –

```
@section TestSection{  
    <h1>Test Content</h1>  
}
```

Q12: What are `NonAction` methods in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

In MVC all public methods have been treated as **Actions**. So if you are creating a method and if you **do not want to use it as an action** method then the method has to be decorated with `NonAction` attribute as shown below:

```
[NonAction]  
public void TestMethod()  
{
```

```
// Method logic  
}
```

Q13: What are *Scaffold templates* in MVC? ☆☆

Topics: ASP.NET MVC

Answer:

Scaffold templates are used to generate code for basic CRUD operations within your ASP.NET MVC applications against your database with the help Entity Framework.

Microsoft provides a helpful scaffolding engine, powered by T4 templates, that automates creating basic CRUD controllers and views for models in ASP.NET MVC applications that use the Entity Framework. (There are also WebAPI and MVC without Entity Framework scaffolders currently available.)